

Mini-project 1 in Advanced Machine Learning (02460)

by Gustav (s214374), Mathias (s214422), Magnus (s224188) & Jacob (s243569) on March 5, 2026

Part A

Following Tomczak (2024), we train VAEs on binarized MNIST using MLP encoder/decoder networks with two 256-unit hidden layers and a 20-dimensional latent space and ReLU activation. We compare a standard Gaussian prior, a mixture of Gaussians ($K = 10$), and a normalizing flow prior with four RealNVP coupling layers.

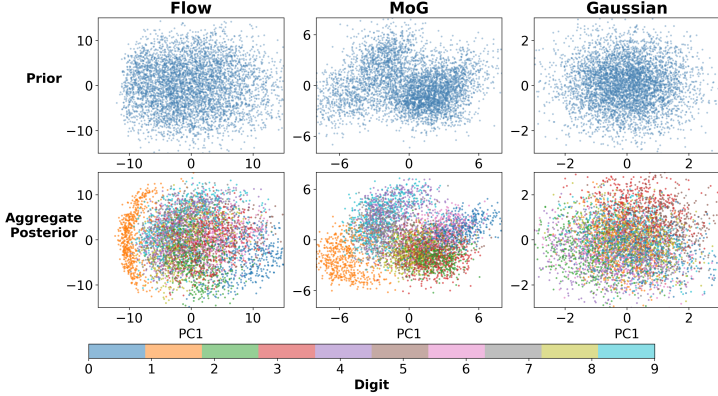


Figure 1: PCA projections of the prior (top) and aggregate posterior (bottom) from 5000 samples. Bottom row colours denote true digit classes.

Figure 1 shows the PCA projections of the aggregate posterior $q(z | x)$ and prior $p(z)$. The standard Gaussian prior forms a compact blob, failing to cover the separated digit clusters of the posterior. The mixture of Gaussians is more flexible and covers more clusters, though some gaps remain. The normalising flow prior is the most expressive, closely matching the non-Gaussian posterior and minimizing the distribution mismatch.

Table 1: Test-set ELBO (nats) on binarised MNIST across 3 runs. Higher is better.

Prior	Mean ELBO $\uparrow$	Std
Standard Gaussian	-83.97	± 0.66
Mixture of Gaussians	-80.45	± 0.58
Normalizing Flow	-77.66	± 0.37

Table 1 shows that the flow prior achieves the best test ELBO, followed by the mixture of Gaussians, while the standard Gaussian performs worst.

Part B

DDPM. A U-Net denoises 28×28 images over $T = 1000$ diffusion steps. **Latent DDPM.** A β -VAE maps images to a $d_z = 20$ latent space where an MLP-based DDPM performs diffusion.

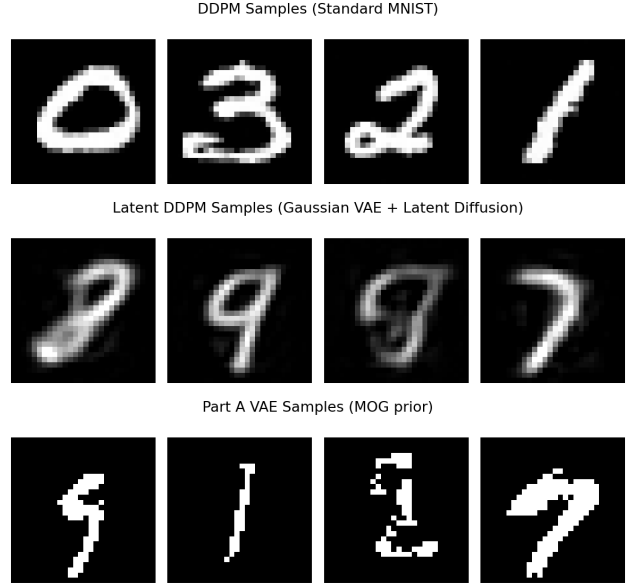


Figure 2: Four samples from DDPM (top), Latent DDPM (middle), and MoG VAE (bottom).

DDPM produces the sharpest digits and the best FID (5.10). Latent DDPM samples are blurrier, while MoG VAE samples appear more binary. The best latent DDPM FID occurs at $\beta = 10^{-2}$, while larger β degrades quality.

Model	FID $\downarrow$	Samples/s
DDPM (image-space)	5.10	8.79
Part-A MoG VAE	18.25	387,672
Latent DDPM $\beta = 10^{-6}$	40.21	
Latent DDPM $\beta = 10^{-4}$	48.80	
Latent DDPM $\beta = 10^{-2}$	39.28	394
Latent DDPM $\beta = 1$	95.29	

Table 2: FID and sampling speed comparison.

The VAE samples extremely fast but with lower quality than DDPM. Latent DDPM is much faster than image-space DDPM but produces blurrier samples due to the VAE decoder.

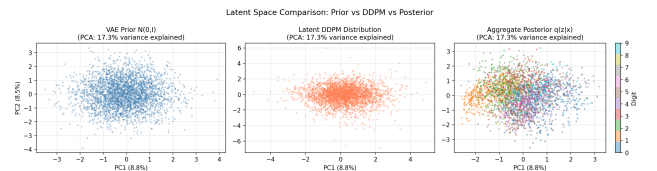


Figure 3: PCA projection of VAE prior, aggregate posterior, and latent DDPM samples.

The VAE prior forms an isotropic Gaussian, while the aggregate posterior shows digit-dependent structure. With large β , the posterior collapses toward the prior, and the latent DDPM samples resemble the Gaussian prior.

Code snippets

VAE ELBO (non-Gaussian prior). Estimated using one Monte Carlo sample with the reparameterisation trick. The three log-terms are summed and averaged.

```
1 def elbo(self, x):
2     q = self.encoder(x)
3     z = q.rsample()                # reparameterisation
4
5     log_p_x_given_z = self.decoder(z).log_prob(x)    # reconstruction
6     log_p_z         = self.prior().log_prob(z)      # works for any prior
7     log_q_z_given_x = q.log_prob(z)                # encoder entropy
8
9     return torch.mean(log_p_x_given_z + log_p_z - log_q_z_given_x)
```

Flow log-density. Using the change-of-variables formula, inputs are mapped to the base Gaussian and corrected by the log-determinant.

```
1 def forward(self, z):
2     s = self.scale_net(z * self.mask)
3     t = self.translation_net(z * self.mask)
4     x = self.mask * z + (1 - self.mask) * (z * torch.exp(s) + t)
5     log_det_J = torch.sum((1 - self.mask) * s, dim=1)
6     return x, log_det_J
7
8 def inverse(self, x):
9     s = self.scale_net(x * self.mask)
10    t = self.translation_net(x * self.mask)
11    z = self.mask * x + (1 - self.mask) * ((x - t) * torch.exp(-s))
12    log_det_J = -torch.sum((1 - self.mask) * s, dim=1)
13    return z, log_det_J
```

Flow log-density. Using the change-of-variables formula, inputs are mapped to the base Gaussian and corrected by the log-determinant.

```
1 def log_prob(self, x):
2     z, log_det_J = self.inverse(x)    # push x to base space
3     return self.base().log_prob(z) + log_det_J
```

DDPM ELBO. Simplified L_{simple} : sample a timestep t and noise ε , diffuse x_0 to x_t , and minimise the MSE between predicted and true noise.

```
1 def elbo(self, x0):
2     t = torch.randint(0, self.T, (B,), device=x0.device)
3     x_t = (self.sqrt_alpha_bars[t]*x0
4            + self.sqrt_one_minus_alpha_bars[t]*eps)
5     eps_pred = self.unet(x_t, t)
6     return F.mse_loss(eps_pred, eps)
```

DDPM sampling. Ancestral reverse diffusion where the U-Net predicts noise and Gaussian noise is added for $t > 0$.

```
1 for t_idx in reversed(range(self.T)):
2     eps_pred = self.unet(x, t)
3     coeff = (1 - alpha) / (1 - alpha_bar).sqrt()
4     x_mean = (x - coeff * eps_pred) / alpha.sqrt()
5     if t_idx > 0:
6         x = x_mean + beta.sqrt() * torch.randn_like(x)
7     else:
8         x = x_mean
```

Latent DDPM training. Encoder means are collected and used to train an MLP noise predictor with the same objective in the 20-dimensional latent space.

```
1 latents = collect_latents(vae, train_loader) # encode to mu
2 for (z0,) in latent_loader:
3     loss = latent_ddpm(z0)    # MSE(eps_pred, eps) on latents
4     loss.backward()
```

Latent DDPM sampling (DDIM, $\eta = 0$). Deterministic reverse diffusion in latent space followed by decoding.

```
1 z = torch.randn(n, latent_dim)    # start from prior
2 for t_curr, t_prev in zip(tau[:-1], tau[1:]):
3     eps_pred = noise_predictor(z, t_curr)
4     pred_z0 = (z - sqrt(1-abar)*eps_pred) / sqrt(abar)
5     z = sqrt(abar_prev)*pred_z0 + sqrt(1-abar_prev)*eps_pred
6 x = vae.decode(z)                  # single decoder pass
```

Declaration of use of generative AI

This declaration **must** be filled out and included as the 3rd page of the document. The questions apply to all parts of the work, including research, project writing, and coding.

- I/we have used generative AI tools: yes

If you answered *yes*, please complete the following sections. List the generative AI tools you have used:

- Gemini, TexGPT % Claude

Describe how the tools were used:

What did you use the tool(s) for? Gemini was used as sparing partner, where we could get feedback and progress reports on our ideas on how to use different solutions.

Claude was used mainly with Github Copilot to help ensure consistency in our code base and link our different functions together whilst giving an overview of changes.

TexGPT was used in overleaf for latex formalia questions.

At what stage(s) of the process did you use the tool(s)? Different tools at different stages.

How did you use or incorporate the generated output? We mostly used the generated output to gain understanding and avoiding too many google searches instead.

References

J. M. Tomczak. *Deep Generative Modeling*. Springer, 2nd edition, 2024.